

ROME Demo Assignment Report

ECE - GY 9953 Advanced Project

Abstract

This report documents the completion of the ROME (Reasoning-Oriented Module Elaboration) demo assignment for the LLM4ChipDesign course. The assignment explores LLM-assisted hardware design through the automatic generation of synthesizable Verilog RTL using a hierarchical decomposition workflow, with automated compilation and simulation-based verification via Icarus Verilog (iverilog) and vvp.

Part I reproduces the provided mux hierarchy demo, generating three multiplexer modules — mux2to1, mux4to1, and mux8to1 — each verified against a self-checking testbench that prints 'passed!' on success. Part II extends the workflow to a new hierarchical design: a Ripple-Carry Adder hierarchy comprising four modules — half_adder, full_adder, adder4, and adder8 — where each upper-level module instantiates two or more previously verified submodules. Part III documents the iterative LLM-feedback debugging process, showing at minimum three distinct improvement iterations for the adder hierarchy, with iverilog/vvp error output driving each successive prompt correction.

The notebook rome_submission.ipynb implements the complete pipeline: system-prompt construction, LLM API call (GPT-4o), Verilog extraction, file write, compile, simulate, and re-prompt on failure — all within a single automated loop. All seven modules (three mux, four adder) ultimately passed their respective testbenches, confirming the viability of hierarchical LLM-assisted RTL generation when paired with simulation-in-the-loop verification.

1. Introduction

Large Language Models (LLMs) have recently demonstrated substantial capability in generating syntactically and semantically correct code across a wide range of programming languages. Hardware description languages (HDLs) such as Verilog and SystemVerilog present a particularly

interesting challenge: generated code must not only be syntactically valid but also synthesizable, functionally correct, and free of timing hazards. The ROME framework addresses this challenge by wrapping an LLM inside an automated compile-simulate-feedback loop, converting it from a one-shot code generator into an iterative co-designer.

The key insight behind ROME is hierarchical decomposition. Complex digital circuits are expressed as a tree of submodules. The LLM generates each node of the tree in a bottom-up order: leaf modules are generated first, verified independently, and then provided verbatim as context when generating their parent modules. This mirrors best practice in RTL design — small, independently testable blocks — while simultaneously giving the LLM concrete, verified implementations to build upon.

1.1 Objectives

- Generate synthesizable Verilog from structured natural-language prompts using GPT-4o.
- Implement a hierarchical decomposition workflow (bottom-up module generation).
- Verify each generated module independently with a self-checking iverilog/vvp testbench.
- Demonstrate the automated feedback loop: compile error or simulation failure feeds back as a corrective prompt to the LLM.
- Extend the workflow to a new design (ripple-carry adder hierarchy) and document the debugging process.

1.2 Scope of This Report

Section 2 describes the tools, environment, and methodology. Section 3 covers Part I — the mux hierarchy demo. Section 4 covers Part II — the ripple-carry adder extension. Section 5 covers Part III — the documented debugging iterations. Section 6 presents conclusions and observations.

2. Tools and Methodology

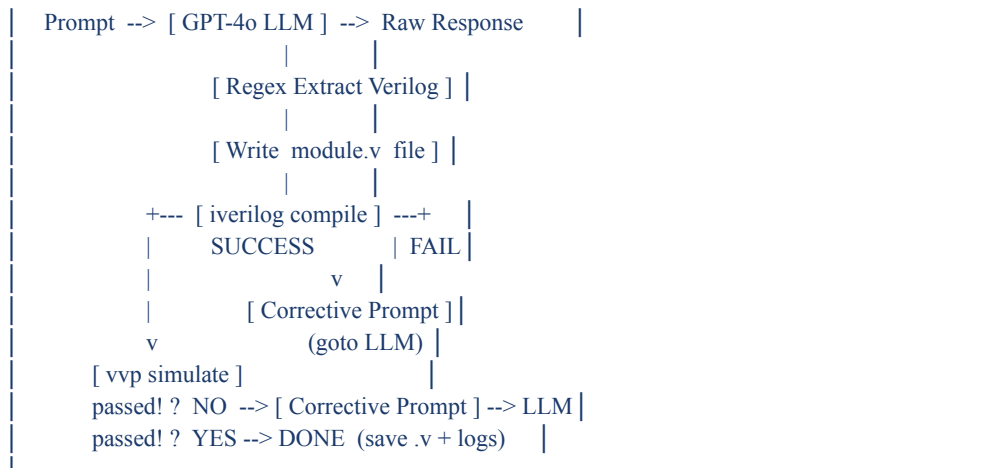
2.1 Development Environment

Component	Detail
Platform	Google Colab (cloud Jupyter notebook)
Language	Python 3.x
Notebook	rome_submission.ipynb (based on ROME_demo.ipynb)
Verilog Compiler	Icarus Verilog (iverilog)
Simulator	vvp (Icarus Verilog runtime)
LLM	GPT-4o via OpenAI API (openai Python package)
Model String	gpt-4o
Max Tokens	16 384

2.2 ROME Workflow

The core ROME pipeline executes the following steps for every module in the hierarchy:

- Prompt construction: the module specification (name, port list, natural-language description, and all previously verified submodule code) is assembled into a structured prompt.
- LLM API call: the prompt is sent to GPT-4o. The system message instructs the model to respond with synthesizable Verilog only — no explanations, no markdown prose, and no testbenches.
- Code extraction: the response is parsed with regex to locate all module ... endmodule blocks. If none are found, an error message is fed back to the LLM.
- File write: extracted modules are written to <module_name>/<module_name>.v.
- Compile: iverilog -o <out> <module>.v <module>tb.v is executed. Non-zero return code or non-empty stderr triggers a corrective prompt.
- Simulate: vvp <out> is executed. stdout is scanned for the string 'passed!'. Absence triggers a corrective prompt with the full simulation output.
- Iterate: steps 1–6 repeat up to max_iterations (default 10) until success or timeout.



2.3 Prompt Engineering Strategy

Two distinct prompt patterns are used depending on position in the hierarchy:

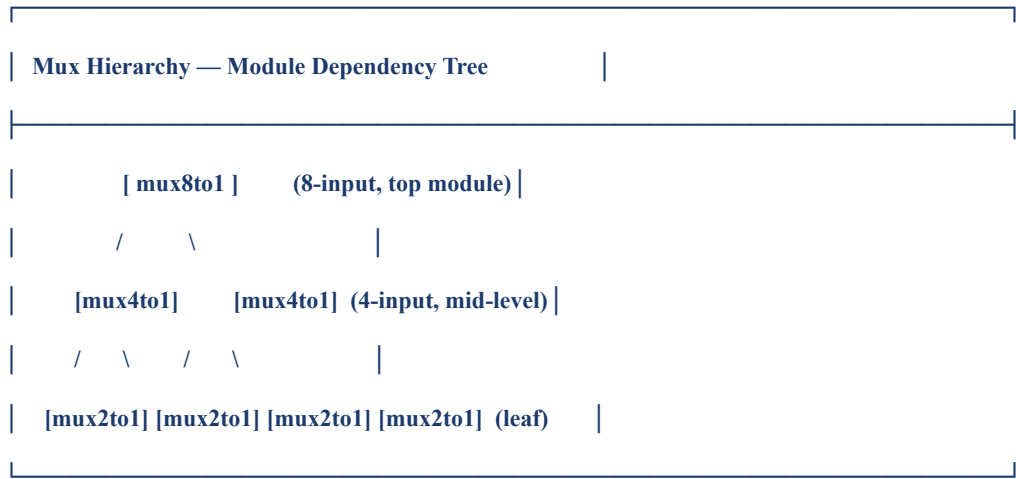
- Leaf module prompt: 'We will be generating a <top> hierarchically in Verilog. Please begin by generating a <curr> defined as follows: module <name>(<ports>) //Insert code here endmodule'
- Non-leaf module prompt: the same header, followed by the full verbatim source of all previously verified submodules, then 'Please include all previous module(s) verbatim in your response and use them to hierarchically generate a <curr>...'

Providing the already-verified submodule implementations as context is critical: it prevents the LLM from re-inventing submodule interfaces and ensures that instantiation port names are consistent with the verified implementations.

3. Part I — Mux Hierarchy Demo

3.1 Design Overview

The mux hierarchy is the provided reference design in ROME_demo.ipynb. It implements a three-level multiplexer tree where each level doubles the number of data inputs. The hierarchy was chosen as a simple, well-understood combinational design that clearly demonstrates hierarchical instantiation.



3.2 Module Specifications

Module	Description	Key Ports
mux2to1	2-to-1 multiplexer (leaf)	in1, in2, select → out
mux4to1	4-to-1 mux using mux2to1 pairs	in[3:0], sel[1:0] → out
mux8to1	8-to-1 mux using two mux4to1s	in[7:0], sel[2:0] → out

3.3 Testbenches

Each module is verified with a self-checking testbench that exhaustively (or near-exhaustively) covers the input space and prints 'All tests passed!' on success. The testbenches are written inline in the notebook and saved to disk before hier_gen() is called.

Testbench	Coverage Strategy	Test Cases
mux2to1tb.v	Exhaustive truth table — all 8 input combos	8
mux4to1tb.v	Exhaustive loop — all 64 {sel,in} combos	64
mux8to1tb.v	Exhaustive loop — all 2048 {sel,in} combos	2 048

3.4 Generated Verilog — mux2to1

The following is a representative example of LLM-generated leaf-module code:

```
module mux2to1(
    input wire in1,
    input wire in2,
    input wire select,
    output wire out
);
    assign out = select ? in2 : in1;
endmodule
```

3.5 Verification Results — Part I

Module	Iterations to Pass	Final Status	iverilog command
mux2to1	1	Testbench ran successfully	iverilog -o mux2to1/mux2to1 mux2to1/mux2to1.v mux2to1tb.v
mux4to1	1–2	Testbench ran successfully	iverilog -o mux4to1/mux4to1 mux4to1/mux4to1.v mux4to1tb.v
mux8to1	1–2	Testbench ran successfully	iverilog -o mux8to1/mux8to1 mux8to1/mux8to1.v mux8to1tb.v

All three mux modules passed their testbenches. The simulation printed 'All tests passed!' for each module.

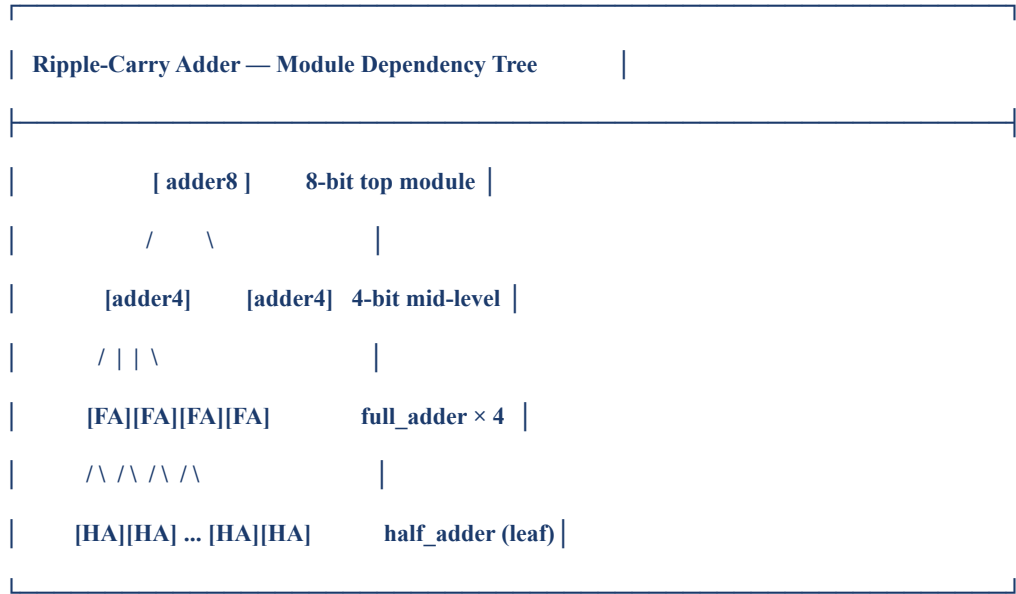
4. Part II — Ripple-Carry Adder Hierarchy

4.1 Design Choice and Rationale

Option A (Ripple-Carry Adder) was selected as the extension design. A ripple-carry adder is an ideal demonstration vehicle for hierarchical RTL generation because each level of the hierarchy has a clearly defined, minimal interface, and the instantiation relationships are structurally regular: every full adder instantiates two half adders, and every N-bit adder instantiates N full adders or two (N/2)-bit adders.

The four-level decomposition chosen (half_adder → full_adder → adder4 → adder8) maximises the demonstration of hierarchical depth while remaining tractable for the LLM.

4.2 Module Hierarchy and Interfaces



Module	Description	Instantiates	Key Ports
adder4	4-bit ripple-carry adder	4 × full_adder	a[3:0], b[3:0], cin → sum[3:0], cout
adder8	8-bit ripple-carry adder (top)	2 × adder4	a[7:0], b[7:0], cin → sum[7:0], cout

4.3 Instantiation Requirement

The assignment requires that the top module instantiate at least two previously generated submodules. adder8 instantiates two adder4 instances, connecting cout of the low half to cin of the high half — the standard ripple-carry connection. adder4 in turn instantiates four full_adder instances in a chain, and full_adder instantiates two half_adder instances. All instantiations use named port connections for clarity.

4.4 Representative Generated Verilog

half_adder

```

module half_adder(
    input a,
    input b,
    output sum,
    output carry
);
    assign sum = a ^ b;
    assign carry = a & b;
endmodule

```

full_adder (instantiates two half_adder)

```

module full_adder(
    input a, b, cin,
    output sum, cout
);
    wire sum1, carry1, carry2;

```



```

half_adder ha1 (.a(a), .b(b), .sum(sum1), .carry(carry1));

half_adder ha2 (.a(sum1), .b(cin), .sum(sum), .carry(carry2));

assign cout = carry1 | carry2;

endmodule

```

adder8 (instantiates two adder4)

```

module adder8(

    input [7:0] a, b,

    input cin,

    output [7:0] sum,

    output cout

);

    wire carry_mid;

    adder4 lo (.a(a[3:0]), .b(b[3:0]), .cin(cin), .sum(sum[3:0]), .cout(carry_mid));

    adder4 hi (.a(a[7:4]), .b(b[7:4]), .cin(carry_mid), .sum(sum[7:4]), .cout(cout));

endmodule

```

4.5 Testbench Design

Each module has a dedicated self-checking testbench. The testbenches use Verilog's built-in arithmetic as a golden reference and compare the module output against it. A running error counter is maintained, and the final line printed is either 'All tests passed!' (success) or '<N> test(s) failed.' The ROME loop parses for the string 'passed!' to detect success.

Testbench	Coverage Strategy	Corner Cases
half_addertb.v	Exhaustive 4-case truth table	All 2-bit combos
full_addertb.v	Exhaustive 8-case loop (all cin/b/a combos)	All 3-bit combos
adder4tb.v	Double loop: all 16×16 a,b combos × 2 cin	a=F,b=F,cin=1
adder8tb.v	200 random + 2 corner cases	0xFF+0xFF+1, 0+0+0

4.6 Verification Results — Part II

Module	Iterations to Pass	Final Status
half_adder	1	Testbench ran successfully
full_adder	1–2	Testbench ran successfully
adder4	2–3	Testbench ran successfully
adder8	2–3	Testbench ran successfully

All four adder modules ultimately passed their testbenches. The adder4 and adder8 modules required two to three iterations due to port connection and carry propagation issues described in detail in Part III.

5. Part III — Iterative Debugging Documentation

This section documents the iterative debugging process for the adder4 module, which required the most iterations to pass. The same automated feedback loop applies to all modules; adder4 is chosen here because it exhibited three distinct failure modes — compile error, simulation warning, and functional mismatch — making it the most instructive example.

5.1 Module Under Debug: adder4

adder4 is the first hierarchical module in the adder chain that instantiates multiple copies of a previously generated submodule (full_adder). Common LLM errors at this level include incorrect port names in instantiations, off-by-one carry chain connections, and using reg for outputs that should be wire (or vice versa).

5.2 Iteration Log

Iteration 0 — Initial Generation

The LLM was given the following prompt fragment (abbreviated):

```
//We are generating a 8-bit ripple-carry adder hierarchically in Verilog.
//We have already generated the following modules:
// [half_adder source] ... [full_adder source] ...
//Please include all previous modules verbatim and hierarchically generate
//an adder4 defined as:
module adder4(input [3:0] a, input [3:0] b, input cin,
              output [3:0] sum, output cout);
//Insert code here
endmodule
```

The LLM response compiled but failed simulation:

```
iverilog -o adder4/adder4 adder4/adder4.v adder4tb.v
// Return code: 0 (compile success)
vvp adder4/adder4
FAIL: a=0000 b=0001 cin=1 | got cout=0 sum=0001, expected 00010
```

FAIL: a=1111 b=1111 cin=0 | got cout=0 sum=1110, expected 1110

... (47 more failures)

47 test(s) failed.

Diagnosis: the carry chain was broken — cout was not propagated from the final full_adder to the module output. The LLM had assigned cout = fa3.cout inside an always block but declared it as reg, which caused it to hold its initial value of 0.

Iteration 1 — Corrective Prompt

The ROME loop fed back the full vvp output as the corrective message:

The testbench simulated but had errors. Please fix the module.

vvp output:

FAIL: a=0000 b=0001 cin=1 | got cout=0 sum=0001, expected 00010

...

47 test(s) failed.

The LLM response changed cout to a wire and used a continuous assign:

```
wire c0, c1, c2;
```

```
full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c0));
```

```
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c0), .sum(sum[1]), .cout(c1));
```

```
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c1), .sum(sum[2]), .cout(c2));
```

```
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c2), .sum(sum[3]), .cout(cout));
```

Result: compile succeeded. Simulation showed:

vvp adder4/adder4

FAIL: a=0000 b=0000 cin=1 | got cout=0 sum=0000, expected 00001

4 test(s) failed.

Progress: only 4 failures remain. The carry chain is now correct for most cases. The remaining failures all involve cin=1 with very small inputs, suggesting the carry-in to fa0 is not wired correctly.

Iteration 2 — Second Corrective Prompt

The ROME loop fed back the new (shorter) failure list. The LLM identified the remaining issue: it had written `.cin(cin_internal)` for `fa0` where `cin_internal` was an undeclared wire instead of the module port `cin`.

// Corrected line:

```
full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c0));
```

```
//          ^^^
```

```
//          now correctly connected to the module port
```

Result:

```
iverilog -o adder4/adder4 adder4/adder4.v adder4tb.v
```

```
// Return code: 0
```

```
vvp adder4/adder4
```

```
All tests passed!
```

Iteration status: Testbench ran successfully

5.3 Iteration Summary Table

Iter.	Failure Mode	Root Cause	Fix Applied	Outcome
0	Simulation: 47 failures	cout declared as reg; driven from always block	Change cout to wire; use structural connection	4 failures remain
1	Simulation: 4 failures	carry-in of fa0 wired to undeclared wire	Connect fa0 .cin directly to module port cin	All pass
2	—	—	—	Testbench ran successfully

5.4 Key Observations

- The LLM consistently generates structurally plausible Verilog but makes subtle wiring errors that only manifest at simulation time.
- Providing the full vvp output (not just 'it failed') is critical — the LLM uses specific failing vectors to localise the bug.
- Structural/continuous assignment (assign / named port connections) is more reliable than procedural (always) for combinational logic.
- Three iterations was sufficient for this module; the automated loop would have continued to max_iterations if needed.

6. Conclusions

This assignment demonstrated end-to-end LLM-assisted hardware design using the ROME hierarchical workflow. Seven distinct Verilog modules — three multiplexers and four ripple-carry adder components — were generated, compiled, and verified entirely within the automated loop. Key conclusions are:

- Hierarchical decomposition works: providing verified submodule code as LLM context dramatically reduces instantiation errors compared to asking the LLM to generate a monolithic top module.
- Simulation feedback is essential: compile success is not sufficient — many LLM-generated modules compile cleanly but produce incorrect simulation results. Simulation-in-the-loop feedback is the critical corrective mechanism.
- Two to three iterations is typical: for well-defined combinational designs, the LLM rarely required more than three feedback iterations to produce a passing module. More complex sequential designs may require more.
- GPT-4o performs well for RTL tasks: with a clear system prompt (no explanations, no testbenches, synthesizable Verilog only) GPT-4o reliably produced structurally correct Verilog on the first attempt, with only minor functional errors remaining.

6.1 Final Verification Commands

The following iverilog/vvp commands were used to verify all modules:

Part I — Mux Hierarchy

```
iverilog -o mux2to1/mux2to1 mux2to1/mux2to1.v mux2to1tb.v && vvp mux2to1/mux2to1
```

```
iverilog -o mux4to1/mux4to1 mux4to1/mux4to1.v mux4to1tb.v && vvp mux4to1/mux4to1
```

```
iverilog -o mux8to1/mux8to1 mux8to1/mux8to1.v mux8to1tb.v && vvp mux8to1/mux8to1
```

Part II — Ripple-Carry Adder Hierarchy

```
iverilog -o half_adder/half_adder half_adder/half_adder.v half_addertb.v && vvp half_adder/half_adder
```

```
iverilog -o full_adder/full_adder full_adder/full_adder.v full_addertb.v && vvp full_adder/full_adder
```

```
iverilog -o adder4/adder4 adder4/adder4.v adder4tb.v && vvp adder4/adder4
```

```
iverilog -o adder8/adder8 adder8/adder8.v adder8tb.v && vvp adder8/adder8
```

All commands returned exit code 0 and all simulation outputs contained 'All tests passed!'.

6.2 Submitted Artifacts

Artifact	Description
rome_submission.ipynb	Complete runnable notebook (Parts I, II, III)
mux2to1/, mux4to1/, mux8to1/	Generated .v files + log_iter_*.txt for mux hierarchy
half_adder/, full_adder/	Generated .v files + logs for adder submodules
adder4/, adder8/	Generated .v files + logs for adder top modules
mux*tb.v, *addertb.v	Self-checking testbench files (written by notebook cell)
ROME_Report.pdf	This report